INF5130 - Algorithmique

Série d'exercices - 3

Exercise 1

La fonction suivante prend en entrée une portion de tableau contenue entre l'indice B_I et l'indice B_S et renvoie deux nombres entiers. Les éléments du tableau sont eux-mêmes des entiers. Vous pouvez supposer que les éléments du tableau sont tous différents les uns des autres et que le nombre d'éléments de la portion de tableau sont tous différents les uns des autres et que le nombre d'éléments de la portion de tableau est pair et plus grand que 0.

```
1  def bidon(tab, bi, bs):
2      if tab[bi] >= tab[bi + 1]:
3          w = tab[bi]
4          x = tab[bi + 1]
5      else:
6          w = tab[bi + 1]
7          x = tab[bi]
8      if  bs <= (bi + 1):
9          n1 = w
10         n2 = x
11     else:
12         (y, z) = bidon(tab, bi + 2, bs)
13         if w >= y:
14             n1 = w
15         else:
16             n1 = y
17         if x <= z:
18             n2 = x
19         else:
20             n2 = z
21         return (n1, n2)
```

a) Supposons que $B_I = 0, B_S = 5$ et que Tab contient les éléments -7, 2, -3, 5, 0 , 1. Faites la trace de l'appel BIDON(TAB, B_I, B_S) : pour chaque appel à BIDON() dans la pile d'exécution, donnez les valeurs de B_I, B_S et les valeurs retournées n_1, n_2, si ces dernières ne sont pas encore connues mettez «?».

b) Décrivez ce que fait BIDON() en général.

c) Soit $T(n)$ le nombre de comparaisons effectués par un appel de BIDON() lorsque le nombre d'éléments de la portion de tableau est égal à n. La comparaison $B_S \leq B_I + 1$ ne compte pas... Écrivez l'équation de récurrence associée à cet algorithme.

d) Résolvez l'équation de récurrence que vous avez trouvée à la question précédente. Vous pouvez la résoudre de manière exacte.

Exercise 2

Voici un algorithme qui permet de résoudre le problème des tours de Hanoi.

```
1 def deplacer(n, t1, t2, t3):
2     if n == 1:
3         print("Déplacler un disque de la tour t1 à la tour t2")
4     else:
5         deplacer(n - 1, t1, t3, t2)
6         print("Déplacer un disque de la tour t1 à la tour t2")
7         deplacer(n - 1, t3, t2, t1)
```

DEPLACER(n, T_1, T_2, T_3) a pour effet d'afficher tous les les déplacements requis pour enfiler n disques sur la tour T_2. Au départ les disques sont enfilés sur la tour T_1.

a) Soit $T(n)$, le nombre de déplacements effectués, ou ce qui revient au même, le nombre de message affichés par la procédure DEPLACER lorsqu'elle est appelée avec n comme paramètre. En supposant que l'affichage d'un message prend un temps égal à 1, écrivez l'équation de récurrence associée à $T(n)$.

b) Résolvez l'équation de récurrence trouvée à la question précédente. Vous pouvez la résoudre de manière exacte.

Exercise 3

Soit dans une matrice M composée d'entier, un élément est considéré comme le maximum global si et seulement s'il est le plus grand élément de la matrice.

a) Écrivez un algorithme **récursif** permettant de résoudre le problème.

b) Donnez l'équation de récurrence de votre algortihme.

c) Donnez une estimation de la complexité de votre équation de récurrence.

Exercise 4

Écrivez un algorithme efficace pour calculer la racine carrée entière d'un entier $n \geq 2$. La racine carrée entière d'un entier n supérieur ou égal à 2. La racine carrée entière de n est le plus grand entier positif r tel que $r^2 \leq n$. Par «algorithme efficace», on entend un algorithme dont le temps d'exécution est proportionnel à $\log n$.

Exercise 5

Le but de cet exercice est de résoudre des équations de récurrence de la forme,

$$T(n) = T(n - \beta) + T(\beta) + n$$

où n est un nombre naturel positif.

a) Résolvez cette équation de manière exacte dans le cas où β est égal à 1. Vous pouvez supposer que $T(1)$ est une constante connue.

b) Résolvez cette équation de manière exacte dans le cas où β est égal à 2. Vous pouvez supposer que $T(2)$ et $T(1)$ sont des constantes connues.

c) Pouvez-vous «deviner» la complexité de la solution lorsque β est un nombre quelconque, mais fixe ?

Exercise 6

Pour chacune des équations de récurrences suivantes, trouvez une fonction f aussi simple que possible telle que $T \in \Theta(f)$

a) $T(n) = 7T(\frac{n}{2}) + n^3$

b) $T(n) = 4T(\frac{n}{3}) + n$

c) $T(n) = nT(n-1) + 1$

d) $T(n) = 8T(\frac{n}{2}) + n^3$

e) $T(n) = 2T(n-1) + n^2$

g) $T(n) = 2T(\frac{n}{3}) + \log n^2$

h) $T(n) = 4T(\frac{n}{3}) + n^{\log_3 4}$

i) $T(n) = 7T(\frac{n}{3}) + n^2 \log n^3$

j) $T(n) = 5T(\frac{n}{3}) + n^{\frac{3}{2}}$

Exercise 7

Le temps d'exécution de l'algorithme A est donné par l'équation de récurrence

$$T_A(n) = 7T_A(\frac{n}{2}) + n^2$$

, alors que celui de l'algorithme A' est donné par l'équation

$$T_{A'} = cT'_A(\frac{n}{4}) + n^2$$

(où c est un entier positif inconnu). Supposez que $T_{A'} \in o(T_A)$. Quelle est la plus grande valeur possible pour c ?

Exercise 8

Résolvez les équations de récurrences ci-bas **sans** utiliser le théorème général.

a) $T(n) = 2T(\frac{n}{3}) + n$

b) $T(n) = 4T(\frac{n}{2}) + \sqrt{n}$

c) $T(n) = 2T(\frac{n}{2}) + n$

d) $T(n) = 2T(\frac{n}{3}) + \log n$

e) $T(n) = 4T(\frac{n}{4}) + n \log n$